

PROJETO INTEGRADOR: SABOR & CLIC

Sistema de Gestão para Dark Kitchens e Espaços Gastronômicos

30 de julho de 2026

1. Introdução e Fluxo do Projeto

Imagine que sua equipe de desenvolvimento foi contratada para criar o núcleo tecnológico da Sabor & Clic, uma startup focada na gestão de Dark Kitchens (cozinhas exclusivas para entregas e eventos) e locação de bancadas/espços gastronômicos equipados.

O objetivo é desenvolver um sistema que opere através de um fluxo de dados tripartido, garantindo a modularidade, o desacoplamento e a escalabilidade da aplicação:

- Interface e Interatividade (Frontend / JavaScript):** As interações dos usuários são capturadas na interface gráfica, validadas e estruturadas via JavaScript Moderno através de Orientação a Objetos.
- Comunicação e Transporte (JSON / Fetch API):** O JavaScript empacota os dados em formato padronizado JSON e realiza requisições assíncronas (HTTP/AJAX via Fetch API) para os endpoints da aplicação.
- API de Backend e Persistência (Flask / Python / SQL):** A camada em Flask recebe as requisições, processa as regras de negócio, aplica segurança/autenticação e persiste as transações diretamente no Banco de Dados Relacional, devolvendo respostas padronizadas em JSON para atualização dinâmica da interface.

2. Contexto e História

A "Sabor & Clic" nasceu para atender pequenos chefs, empreendedores marmiteiros e redes de delivery que precisam de infraestrutura de ponta sem os custos de um restaurante tradicional. No entanto, ficha técnica de pratos, controle de pedidos e agendamento de horários das bancadas ainda é realizado por meio de planilhas e anotações manuais. Isso tem gerado falhas graves: erros de precificação, conflitos de horários na locação das cozinhas e atrasos no preparo de pedidos. Para continuar expandindo, a startup precisa de um sistema web/back-end eficiente que automatize o processamento de pedidos, agendamento de bancadas e criação de cardápios.

3. Atores e Funcionalidades Detalhadas

3.1 Administrador (Gestor do Catálogo)

- Características (Dados):** ID de identificação, Nome Completo, Email de acesso, Nível de permissão (Administrador).
- Funcionalidades:**
 - Gerenciamento de Cardápio:** Criar, editar, atualizar e remover itens do catálogo (CRUD).
 - Controle:** Definir preços, descrições e ativar/inativar pratos para exibição.

3.2 Cliente (Consumidor)

- Características (Dados):** ID de identificação, Nome, Email, Endereço de entrega, Histórico de pedidos realizados.
- Funcionalidades:**

- **Visualização de Menu:** Acessar a lista de pratos disponíveis em tempo real.
- **Gestão de Pedidos:** Montar carrinho com itens selecionados e finalizar a compra.
- **Rastreamento:** Consultar o status atual do seu pedido (Ex: Aguardando, Em Preparo, Pronto).

3.3 Cozinheiro (Operador de Produção)

- **Características (Dados):** ID de identificação, Nome, Turno de trabalho, Especialidade na cozinha.
- **Funcionalidades:**
 - **Gestão de Fila:** Visualizar a fila de pedidos pendentes em tempo real.
 - **Agendamento de Preparo:** Definir e registrar o tempo estimado de produção para cada item ou pedido.
 - **Atualização de Status:** Alterar o status do pedido para 'Iniciando Preparo' ou 'Pronto para Entrega', permitindo o fluxo logístico.

4. Arquitetura, Desenvolvimento e Reflexões

A infraestrutura técnica é baseada nos seguintes pilares:

- **Interface (HTML/CSS3 Puro):** Estrutura semântica, acessível e responsiva desenhada sob medida.
- **Lógica de Apresentação e POO (JavaScript / DOM / Fetch):** O JavaScript processa eventos de interface, valida regras de entrada via Classes (ES6+) com encapsulamento, serializa dados em JSON e despacha requisições HTTP assíncronas.
- **API Backend (Flask):** Camada de inteligência (de negócio) em Python responsável pelas regras de negócio, segurança (sessões/cookies), rotas RESTful e sanitização.
- **Protocolo de Comunicação (JSON):** Protocolo (formato universal e exclusivo) de intercâmbio de dados entre o Front-end e o Back-end (Node.js e Flask).
- **Conexão ao Banco de Dados:** Exclusiva do Flask, garantindo a integridade dos dados.

Perguntas para Reflexão:

1. Como o uso de Orientação a Objetos no JavaScript / Frontend facilita a escalabilidade das rotas e a manutenção do código frente a futuras funcionalidades?
2. De que maneira a comunicação via JSON entre o Front-end (JavaScript) e a API (Flask) isola as responsabilidades de interface e processamento, e quais os riscos de latência nessa integração?
3. Como a centralização da conexão ao banco de dados no Flask impacta a segurança dos dados e a performance em relação à validação das regras de negócio?
4. Considerando que cada ator tem uma função única, como o sistema deve tratar conflitos se um cliente solicitar um prato que o administrador removeu simultaneamente do cardápio?

5. Requisitos por Disciplina

5.1 Artes Visuais e Design

Responsável pela criação do ecossistema visual do sistema. Inclui prototipação de alta fidelidade, design de interface (UI/UX), escolha de tipografia, paleta de cores e criação do painel de pedidos (Kitchen Display System).

- **Parte 1 (3º Bimestre):** Wireframes e protótipo interativo no Figma; Definição da identidade visual (paleta de cores RGB/HEX, hierarquia tipográfica, contraste acessível WCAG); Guia de Estilos (Design System básico); Layout do Kitchen Display System (KDS) e Cardápio Digital.
- **Parte 2 (4º Bimestre):** Implementação do front-end fiel ao protótipo; Estruturação semântica em HTML5; Estilização responsiva em CSS3 (Flexbox/CSS Grid, Media Queries); Aplicação dos padrões de usabilidade e feedback visual de interação. **Espaço do Professor:** [Preencher conteúdos específicos, critérios de avaliação e datas].

Espaço do Professor

- **PROIBIÇÃO DE PLATAFORMAS PRONTAS / NO-CODE:** É estritamente vedado o uso de criadores de sites automatizados ou plataformas visuais prontas (como Wix, WordPress, Squarespace, Webflow, Shopify ou geradores automáticos de código do Figma). Todo o código de interface deve ser digitado e construído artesanalmente pelo grupo utilizando HTML5 e CSS3 puros.
- **Fidelidade ao Protótipo:** A interface final implementada em código será avaliada pela correspondência direta com o protótipo aprovado no Figma durante o 3º bimestre.
- **Critérios de Avaliação:** Clareza na hierarquia visual, consistência cromática, legibilidade tipográfica, responsividade para diferentes resoluções (Mobile/Desktop) e organização do código CSS.

5.2 Linguagem de Programação — JavaScript (SPOWEB1)

Responsável pela inteligência da interface, manipulação dinâmica do DOM, validação de regras de negócio no cliente, modelagem em Orientação a Objetos e comunicação assíncrona.

- **Parte 1 (3º Bimestre):** Lógica fundamental (estruturas de repetição modernas for...of, for...in, forEach, arrays e objetos literais); Manipulação dinâmica do DOM (criação e atualização de elementos da tela do cardápio/carrinho); Validação de formulários e cálculo em memória de subtotais e pedidos.
- **Parte 2 (4º Bimestre):** Refatoração com Orientação a Objetos (Classes ES6+, herança, métodos estáticos e encapsulamento real com campos privados #); Tratamento de exceções com try...catch; Comunicação assíncrona com a API Flask através da Fetch API (async/await, métodos GET/POST/PUT/DELETE); Serialização e desserialização de payloads JSON para envio e recebimento de dados; Gerenciamento do estado da aplicação e atualização dinâmica das tabelas/painéis sem recarregar a página.

Espaço do Professor

- **Aderência aos Padrões ES6+:** O código deve utilizar sintaxe moderna (const/let, arrow functions, template literals, destructuring e métodos funcionais de array como map, filter e reduce).
- **Modelagem Orientada a Objetos:** É obrigatória a criação de classes para estruturar entidades do sistema no front-end (ex: `Prato`, `ItemPedido`, `Carrinho`, `Pedido`, `Usuario`), utilizando atributos privados (`#`) e métodos de manipulação de estado.
- **Integração e Assincronismo:** O front-end não deve conter código de acesso a banco de dados nem recarregar a página (F5) para atualizar a tela. Todas as requisições para a API Flask devem ser assíncronas via `fetch()` encapsuladas em blocos `try...catch`, tratando estados de carregamento (loading), sucesso e erros de resposta HTTP.
- **Boas Práticas (Clean Code):** Modularização de scripts em arquivos separados por responsabilidade (ex: `models.js`, `api.js`, `app.js`), nomenclatura clara de funções e variáveis, e código sem erros no console (F12).

5.3 Linguagem de Programação — Python (API Backend & Persistência)

Desenvolvimento da camada de serviço (API) utilizando o framework Flask para o processamento de regras de negócio e persistência de dados.

- **Parte 1 (3º Bimestre):** Configuração do ambiente, definição da estrutura de arquivos baseada no padrão MVC (Model-View-Controller) e desenvolvimento de rotas (endpoints) fundamentais. Foco na serialização de dados para JSON e integração inicial com o servidor JavaScript / Frontend.
- **Parte 2 (4º Bimestre):** Implementação de funcionalidades avançadas: uso de sessões e cookies para gerenciamento de autenticação, definição de middlewares para interceptação de requisições e uso de Flash Messages para feedback ao usuário. Integração robusta com Banco de Dados e segurança nas rotas.

Espaço do Professor:

- Critérios de avaliação baseados nos conceitos da disciplina SPOLBP1: organização do código, modularização e aderência aos padrões arquiteturais propostos.
- É obrigatório o uso de arquitetura MVC: separar logicamente a definição dos modelos, o controle das rotas e a apresentação (mesmo que a API retorne apenas JSON).
- Implementação de segurança: Utilizar cookies e sessões para autenticação e middlewares para garantir que apenas usuários autorizados acessem determinadas rotas da API.
- Feedback: O sistema deve prover feedback visual claro (Flash Messages) após operações críticas como login, cadastro ou erro de validação.

5.4 Banco de Dados

Modelagem e implementação da camada de dados do sistema.

- **Etapa 1 (3º Bimestre):** Modelo Conceitual (DER no brModelo) e Dicionário de Dados em .docx.
- **Etapa 2 (4º Bimestre):** Modelo Físico SQL completo, Triggers para baixa automática de estoque e queries de relatórios.

Espaço do Professor: (Banco de Dados): [Aguardando validação dos Profs. Francisco e Matheus para a Versão 3]

5.5 Segurança

Definição de permissões de acesso e regras de validação no back-end e front-end.

- **Parte 1 (3º Bimestre):** Definição dos níveis de acesso (RBAC - Role-Based Access Control) e regras de validação de entrada de dados.
- **Parte 2 (4º Bimestre):** Implementação de autenticação segura e sanitização de entradas (prevenção de SQL Injection).

Espaço do Professor

Utilizar o guia **OWASP** Top-10, que são mencionados nos itens as técnicas de segurança para implantação em aplicações Web. Avaliação: cada bimestre (Parte 1 e 2) **2** pontos/apresentados e entregues nas respectivas datas **08/9** e **03/11**.

6. Desenvolvimento e Entregas

6.1 Parte 1: Estrutura, Documentação e Back-end Simulado

- **Foco:** Planejamento, protótipo navegável no Figma (AVDI), documentação conceitual do BD (DER), lógica inicial de rotas Flask e scripts JS com manipulação de DOM.
- **Data de Entrega:** 10/09/2026 até as 23h55.

6.2 Parte 2: Orientação a Objetos, Banco de Dados Relacional e Triggers

Foco: Front-end responsivo codificado (HTML/CSS), refatoração JS com POO e Fetch API, persistência completa via Flask no Banco de Dados Relacional e Triggers ativas.

- **Data de Entrega:** 07/11/2026 até as 23h55.
- **Apresentações:** Turma 231 – 10/11 e 17/11; Turma 213 – 12/11 e 19/11

7. Apêndice A — Estrutura e Entregáveis de Banco de Dados

A entrega de Banco de Dados deve seguir rigorosamente a organização de arquivos SQL para garantir a reprodutibilidade do ambiente:

Arquivo	Descrição
01_modelo_fisico.sql	Scripts de criação de tabelas, chaves e restrições.
02_insercoes_basicas.sql	Dados mínimos para funcionamento do sistema.
11_queries_relatorios.sql	Consultas analíticas (vendas, estoque, ocupação).
20_triggers.sql	Automação de regras de negócio (ex: baixa de estoque).

8. Considerações Finais

1. A documentação escrita deve seguir as normas **ABNT**.
2. A entrega no Moodle deve conter: Apresentação (PDF), Documentação (PDF) e Projeto (.zip ou link do repositório GitHub com commits de todos os integrantes).
3. **Qualidade de Código:** Serão avaliados legibilidade, nomenclatura semântica de variáveis/funções, indentação consistente e ausência de trechos de código comentados ou redundantes.
4. **Política de Prazos:** Entregas atrasadas terão decréscimo de **20% na nota** por uma semana; após isso, não serão aceitas.

Profª. Claudete de Oliveira Alves

Local e data: São Paulo, 31 de agosto de 2026

Documento atualizado 31/08/2026. As informações contidas são de responsabilidade do solicitante.